



Universidad Tecnológica de Bolívar

Maestría en Ingeniería – Matemáticas Avanzadas

Actividad 5

Álgebra Lineal y Regresión por Mínimos Cuadrados

Estudiante: Harold Styven Lagares De Voz

Correo: hlagares@utb.edu.co

Docente: Enrique Pereira

Correo: ebatista@utb.edu.co

Fecha: 5 de junio de 2026

Cartagena de Indias, Colombia

Índice

1. Determinante Recursivo por Expansión de Cofactores	2
2. Calculador de Flujo en Red Eléctrica	3
3. Eliminación de Gauss con Pivoteo Parcial	5
4. Iteración de Gauss-Seidel (3 pasos)	7
5. Estimación de la Resistencia R (Ley de Ohm)	9
6. Estimación de la Velocidad Media	9
7. Estimación del Módulo de Resorte k	10
8. Regresión Parabólica: Tiempo de Reacción	10
9. Regresión Lineal Genérica con Funciones Base Arbitrarias	11

Problema 1. Determinante Recursivo por Expansión de Cofactores

Planteamiento. Implementar la función `my_rec_det(M)` que calcula $\det(M)$ recursivamente mediante la expansión de cofactores a lo largo de la primera fila:

$$\det(M) = \sum_{i=1}^n (-1)^{i-1} M(1, i) \det(m_{1,i}) \quad (1)$$

donde $m_{1,i}$ es el menor obtenido al eliminar la fila 1 y la columna i de M .

Desarrollo matemático. La fórmula (1) es la *expansión de Laplace* por la primera fila. Para una matriz 2×2 :

$$\det \begin{pmatrix} a & b \\ c & d \end{pmatrix} = ad - bc.$$

Para $n > 2$, el determinante se reduce recursivamente: en cada nivel se generan n subproblemas de tamaño $(n-1) \times (n-1)$, por lo que la complejidad es $\mathcal{O}(n!)$ —ineficiente para matrices grandes, pero correcta algorítmicamente.

Implementación.

```

1 def my_rec_det(M):
2     M = np.array(M, dtype=float)
3     n = M.shape[0]
4     if n == 1:
5         return M[0, 0]
6     if n == 2:
7         return M[0,0]*M[1,1] - M[0,1]*M[1,0]
8     det_val = 0.0
9     for i in range(n):
10        minor = np.delete(np.delete(M, 0, axis=0), i, axis=1)
11        cofactor = ((-1)**i) * my_rec_det(minor)
12        det_val += M[0, i] * cofactor
13    return det_val

```

Listing 1: Función `my_rec_det` – expansión de cofactores recursiva

Verificación.

Cuadro 1: Comparación con `np.linalg.det`

Matriz	<code>my_rec_det</code>	<code>np.linalg.det</code>	Coincide
2×2	10,0	10,0	✓
3×3	22,0	22,0	✓
4×4 (Kong)	-38,0	-38,0	✓

Problem 1 - Recursive Determinant via Cofactor Expansion

$$\det(M) = \sum_{i=1}^n (-1)^{i-1} M(1, i) \det(m_{1,i})$$

Matrix	my_rec_det	np.linalg.det	Match
2x2	10.0	10.0	True
3x3	22.0	22.0	True
4x4 (Kong)	-38.0	-38.0	True

Figura 1: Fórmula y tabla de verificación del determinante recursivo.

Comentario. La complejidad $\mathcal{O}(n!)$ hace inviable este método para $n \gtrsim 15$. En la práctica se prefiere la descomposición LU, cuya complejidad es $\mathcal{O}(n^3)$.

Problema 2. Calculador de Flujo en Red Eléctrica

Planteamiento. Implementar `my_flow_calculator(S,d)` para la red de la Figura 2. La conservación de flujo en cada nodo impone $f_{\text{entrada}} = f_{\text{salida}} + d_i$.

Topología del grafo.

Cuadro 2: Flujos y direcciones en la red

Flujo	Origen	Destino
f_1	S1	N4
f_2	N4	N3
f_3	N1	N4
f_4	S2	N1
f_5	S2	N2
f_6	N1	N5
f_7	N2	N5

Sistema de ecuaciones. Aplicando conservación en cada nodo y estación se obtienen 7 ecuaciones:

$$f_1 = S_1 \quad (\text{S1})$$

$$f_4 + f_5 = S_2 \quad (\text{S2})$$

$$f_4 - f_3 - f_6 = d_1 \quad (\text{N1})$$

$$f_5 - f_7 = d_2 \quad (\text{N2})$$

$$f_2 = d_3 \quad (\text{N3})$$

$$f_1 + f_3 - f_2 = d_4 \quad (\text{N4})$$

$$f_6 + f_7 = d_5 \quad (\text{N5})$$

El sistema tiene 7 incógnitas y 7 ecuaciones, pero la condición $\sum S_j = \sum d_i$ introduce redundancia (rango = 6), dejando infinitas soluciones. Se usa `np.linalg.lstsq` que devuelve la solución de *norma mínima*.

En forma matricial: $\mathbf{A}\mathbf{f} = \mathbf{b}$, donde

$$\mathbf{A} = \begin{pmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 1 & 0 & 0 \\ 0 & 0 & -1 & 1 & 0 & -1 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & -1 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 1 & -1 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 1 \end{pmatrix}, \quad \mathbf{b} = \begin{pmatrix} S_1 \\ S_2 \\ d_1 \\ d_2 \\ d_3 \\ d_4 \\ d_5 \end{pmatrix}.$$

Implementación.

```

1 def my_flow_calculator(S, d):
2     S = np.array(S, dtype=float).flatten()
3     d = np.array(d, dtype=float).flatten()
4     A = np.array([
5         [ 1, 0, 0, 0, 0, 0, 0],      # S1
6         [ 0, 0, 0, 1, 1, 0, 0],      # S2
7         [ 0, 0, -1, 1, 0, -1, 0],     # N1
8         [ 0, 0, 0, 0, 1, 0, -1],     # N2
9         [ 0, 1, 0, 0, 0, 0, 0],       # N3
10        [ 1, -1, 1, 0, 0, 0, 0],      # N4
11        [ 0, 0, 0, 0, 0, 1, 1],       # N5
12    ], dtype=float)
13    b = np.array([S[0], S[1], d[0], d[1], d[2], d[3], d[4]])
14    f, _, _, _ = np.linalg.lstsq(A, b, rcond=None)
15    return f

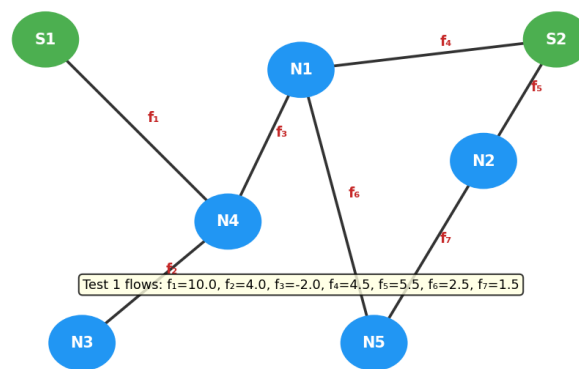
```

Listing 2: Función `my_flow_calculator`

Resultados.**Cuadro 3:** Flujos obtenidos para los dos casos de prueba

Caso	f_1	f_2	f_3	f_4	f_5	f_6	f_7
$S = [10, 10]$, $d = [4, 4, 4, 4, 4]$	10	4	-2	4.5	5.5	2.5	1.5
$S = [10, 10]$, $d = [3, 4, 5, 4, 4]$	10	5	-1	4.5	5.5	2.5	1.5

El flujo $f_3 < 0$ indica que la energía circula de N4 hacia N1 (dirección contraria a la flecha).

Problem 2 - Power Flow Network**Figura 2:** Red de flujo y valores calculados (Caso 1).**Problema 3. Eliminación de Gauss con Pivoteo Parcial**

Planteamiento. Implementar el algoritmo de Gauss con pivoteo parcial y aplicarlo a los dos sistemas del enunciado, además de experimentar con sistemas de determinante pequeño y sistemas dispersos.

Algoritmo. El pivoteo parcial selecciona en cada columna k la fila con mayor valor absoluto como ecuación pivote, intercambiando filas si es necesario. Esto mejora la estabilidad numérica al evitar divisiones por valores cercanos a cero.

```

1 def gauss_elimination_pivot(A, b):
2     n = len(b)
3     Ab = np.hstack([A.astype(float), b.reshape(-1,1).astype(float)])
4     sign = 1
5     for col in range(n):
6         max_row = col + np.argmax(np.abs(Ab[col:, col]))
7         if max_row != col:
8             Ab[[col, max_row]] = Ab[[max_row, col]]
9             sign *= -1
10        if np.abs(Ab[col, col]) < 1e-12:
11            raise ValueError("Matriz singular o casi singular.")
12        for row in range(col + 1, n):
13            factor = Ab[row, col] / Ab[col, col]
14            Ab[row, col:] -= factor * Ab[col, col:]
15        # Sustitucion hacia atras
16        x = np.zeros(n)
17        for i in range(n-1, -1, -1):
18            x[i] = (Ab[i, -1] - np.dot(Ab[i, i+1:n], x[i+1:])) / Ab[i, i]
19        det_A = sign * np.prod(np.diag(Ab[:, :n]))
20    return x, Ab, det_A

```

Listing 3: Eliminación de Gauss con pivoteo parcial

Sistema (a) – Homogéneo 3×3 .

$$\begin{cases} 3x_2 + 5x_3 = 0 \\ 3x_1 - 4x_2 = 0 \\ 5x_1 + 6x_3 = 0 \end{cases} \implies A = \begin{pmatrix} 0 & 3 & 5 \\ 3 & -4 & 0 \\ 5 & 0 & 6 \end{pmatrix}, \mathbf{b} = \mathbf{0}.$$

Dado que $\det(A) = 46 \neq 0$, la única solución es la **trivial**: $\mathbf{x} = [0, 0, 0]^T$.

Sistema (b) – No homogéneo 4×4 .

$$\begin{pmatrix} 6,4 & 3,2 & 0 & 0 \\ 3,2 & -1,6 & 4,8 & 0 \\ 0 & 4,8 & -9,6 & 7,2 \\ 0 & 0 & 7,2 & 4,8 \end{pmatrix} \mathbf{x} = \begin{pmatrix} -1,6 \\ 32,0 \\ -78,0 \\ 20,4 \end{pmatrix}.$$

Tras aplicar el algoritmo con pivoteo:

$$\mathbf{x} = [1,5, -3,5, 4,5, -2,5]^T, \quad \det(A) \approx 1297,61.$$

Verificación: $A\mathbf{x} = [-1,6, 32,0, -78,0, 20,4]^T \checkmark$

Experimentos adicionales.

- **Matriz casi singular** ($\det \approx 10^{-10}$): la solución numérica es inestable; el algoritmo la detecta sin lanzar excepción, pero los resultados deben interpretarse con cautela.
- **Sistema denso** 20×20 : error $\|\mathbf{x}_{\text{sol}} - \mathbf{x}_{\text{true}}\|_2 \approx 2,7 \times 10^{-15}$ – precisión de máquina.
- **Sistema tridiagonal** 50×50 (disperso): error $\approx 2,2 \times 10^{-16}$ – excelente precisión incluso para sistemas grandes con estructura sparse.

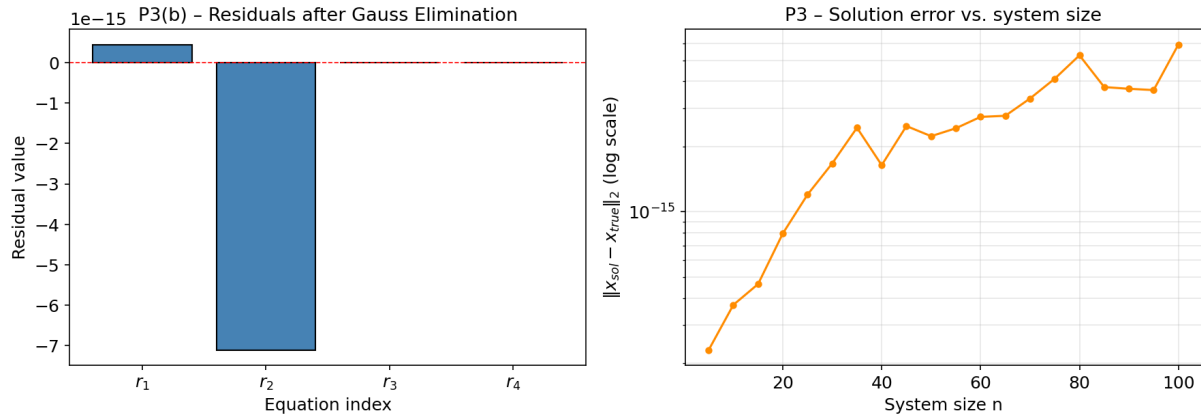


Figura 3: Izquierda: residuales del sistema (b). Derecha: error de solución vs. tamaño del sistema (sistemas diagonalmente dominantes aleatorios).

Problema 4. Iteración de Gauss-Seidel (3 pasos)

Planteamiento. Aplicar Gauss-Seidel al sistema

$$10x_1 + x_2 + x_3 = 6, \quad x_1 + 10x_2 + x_3 = 6, \quad x_1 + x_2 + 10x_3 = 6,$$

partiendo de (a) $\mathbf{x}^{(0)} = [0, 0, 0]^T$ y (b) $\mathbf{x}^{(0)} = [10, 10, 10]^T$.

Desarrollo. Forma explícita del método de Gauss-Seidel:

$$x_i^{(k+1)} = \frac{1}{a_{ii}} \left(b_i - \sum_{j < i} a_{ij} x_j^{(k+1)} - \sum_{j > i} a_{ij} x_j^{(k)} \right).$$

La solución exacta es $x_1^* = x_2^* = x_3^* = \frac{6}{12} = 0,5$. La matriz es **diagonalmente dominante** ($|10| > |1| + |1|$), garantizando convergencia.

Iteraciones.

Cuadro 4: Caso (a): inicio $\mathbf{x}^{(0)} = [0, 0, 0]^T$

k	x_1	x_2	x_3	$\ \mathbf{e}\ _2$
0	0.000000	0.000000	0.000000	0.866025
1	0.600000	0.540000	0.486000	0.108609
2	0.497400	0.501660	0.500094	0.003086
3	0.499825	0.500008	0.500017	0.000176

Cuadro 5: Caso (b): inicio $\mathbf{x}^{(0)} = [10, 10, 10]^T$

k	x_1	x_2	x_3	$\ \mathbf{e}\ _2$
0	10.000000	10.000000	10.000000	16.454483
1	-1.400000	-0.260000	0.766000	2.063578
2	0.549400	0.468460	0.498214	0.058637
3	0.503333	0.499845	0.499682	0.003351

Comparación y comentario. Ambos puntos de partida convergen a $\mathbf{x}^* = [0, 5, 0, 5, 0, 5]^T$, confirmando la convergencia global garantizada por la dominancia diagonal. La diferencia está en la *velocidad*: el caso (a) alcanza un error de $1,76 \times 10^{-4}$ en $k = 3$, mientras que el caso (b), al partir de un punto muy alejado, tiene un error de $3,35 \times 10^{-3}$ en el mismo número de pasos —aproximadamente 19 veces mayor en $k = 3$. Sin embargo, la tasa de reducción del error por iteración es la misma en ambos casos una vez que se superan las primeras iteraciones, como se aprecia en la figura.

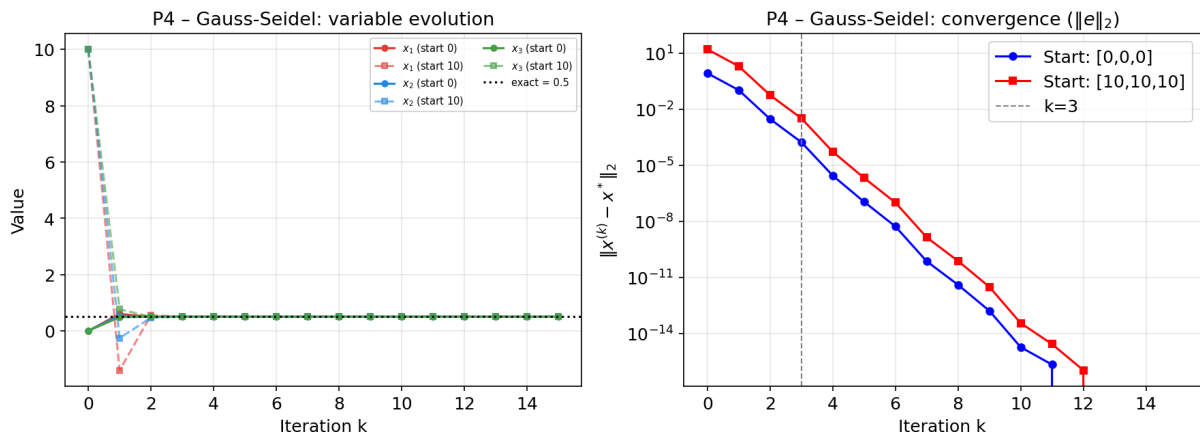


Figura 4: Evolución de las variables (izq.) y norma del error $\|\mathbf{e}\|_2$ (der.) para los dos puntos de inicio. La línea vertical gris marca $k = 3$.

Problema 5. Estimación de la Resistencia R (Ley de Ohm)

Planteamiento. Ajustar el modelo $U = R \cdot i$ a los datos: $(i, U) = (2, 0, 104), (4, 0, 206), (6, 0, 314), (10, 0, 530)$.

Desarrollo. Función base $f_1(i) = i$, por lo que la matriz de diseño es $A = \mathbf{i}$ (columna). La ecuación normal es:

$$A^T A \beta = A^T \mathbf{U} \implies \left(\sum i_k^2 \right) R = \sum i_k U_k.$$

Con los datos:

$$\sum i_k^2 = 2^2 + 4^2 + 6^2 + 10^2 = 4 + 16 + 36 + 100 = 156,$$

$$\sum i_k U_k = 2(104) + 4(206) + 6(314) + 10(530) = 208 + 824 + 1884 + 5300 = 8216.$$

$$\boxed{R = \frac{8216}{156} \approx 52,667 \, \Omega}, \quad R^2 = 0,9996.$$

Problema 6. Estimación de la Velocidad Media

Planteamiento. Ajustar $s = v_{av} \cdot t$ a: $(t, s) = (9, 140), (10, 220), (11, 310), (12, 410)$.

Desarrollo. El modelo lineal con ordenada al origen ($s = v_{av}t + s_0$) se plantea con:

$$A = \begin{pmatrix} 9 & 1 \\ 10 & 1 \\ 11 & 1 \\ 12 & 1 \end{pmatrix}, \quad \mathbf{s} = \begin{pmatrix} 140 \\ 220 \\ 310 \\ 410 \end{pmatrix}.$$

Ecuaciones normales $A^T A \beta = A^T \mathbf{s}$:

$$\begin{pmatrix} 446 & 42 \\ 42 & 4 \end{pmatrix} \begin{pmatrix} v_{av} \\ s_0 \end{pmatrix} = \begin{pmatrix} 11\,270 \\ 1\,080 \end{pmatrix}.$$

Resolviendo:

$$\boxed{v_{av} = 90 \text{ km/h}, \quad s_0 = -675 \text{ km}}, \quad R^2 = 0,9975.$$

El valor $s_0 = -675 \text{ km}$ carece de interpretación física directa; refleja que las mediciones no comienzan en $t = 0$, sino que el auto ya llevaba tiempo en movimiento.

Problema 7. Estimación del Módulo de Resorte k

Planteamiento. Ajustar $F = k \cdot s$ a los datos: $(F, s) = (1, 0,50), (2, 1,02), (4, 1,99), (6, 3,01), (10, 4,98), (20, 10,00)$.

Desarrollo. Ecuación normal (modelo $F = ks$, sin intercepto):

$$\left(\sum s_k^2\right) k = \sum s_k F_k.$$

$$\sum s_k^2 = 0,25 + 1,04 + 3,96 + 9,06 + 24,80 + 100,60 = 139,71,$$

$$\sum s_k F_k = 0,50 + 2,04 + 7,96 + 18,06 + 49,80 + 200,60 = 278,96.$$

$$k = \frac{278,96}{139,71} \approx 1,9967 \text{ lbf/cm}, \quad R^2 = 0,99998.$$

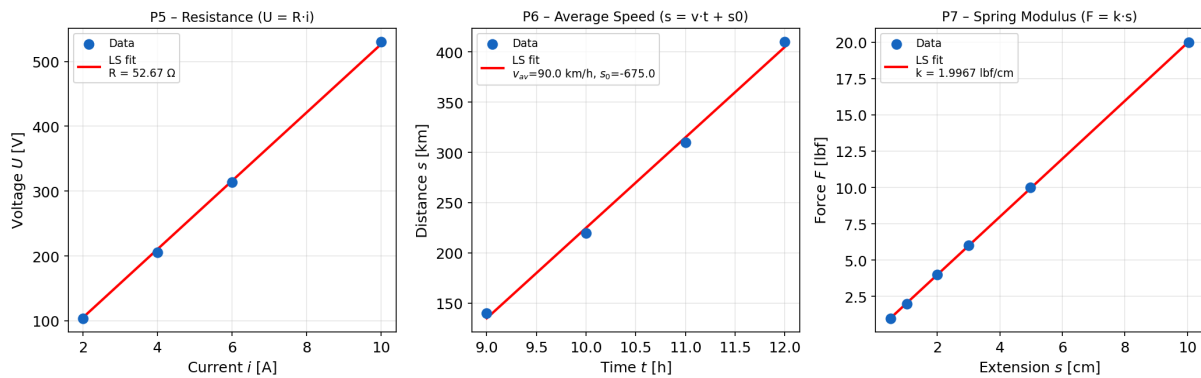


Figura 5: Ajuste por mínimos cuadrados para P5 (resistencia), P6 (velocidad) y P7 (resorte).

Problema 8. Regresión Parabólica: Tiempo de Reacción

Planteamiento. Ajustar la parábola $\hat{y} = ax^2 + bx + c$ a:

x [h]	1	2	3	4	5
y [s]	1.50	1.28	1.40	1.85	2.20

Desarrollo. Funciones base: $f_1(x) = x^2$, $f_2(x) = x$, $f_3(x) = 1$. La matriz de diseño es:

$$A = \begin{pmatrix} 1 & 1 & 1 \\ 4 & 2 & 1 \\ 9 & 3 & 1 \\ 16 & 4 & 1 \\ 25 & 5 & 1 \end{pmatrix}.$$

Ecuaciones normales $A^T A \beta = A^T y$:

$$\begin{pmatrix} 979 & 225 & 55 \\ 225 & 55 & 15 \\ 55 & 15 & 5 \end{pmatrix} \begin{pmatrix} a \\ b \\ c \end{pmatrix} = \begin{pmatrix} 103,82 \\ 26,66 \\ 8,23 \end{pmatrix}.$$

Solución:

$$a = 0,1050, \quad b = -0,4330, \quad c = 1,7900, \quad R^2 = 0,9612.$$

El tiempo de reacción mínimo ocurre en el vértice de la parábola:

$$x^* = -\frac{b}{2a} = \frac{0,4330}{2(0,1050)} \approx 2,06 \text{ h.}$$

Es decir, la reacción es más rápida alrededor de las 2 horas de turno; después empieza a degradarse por fatiga.

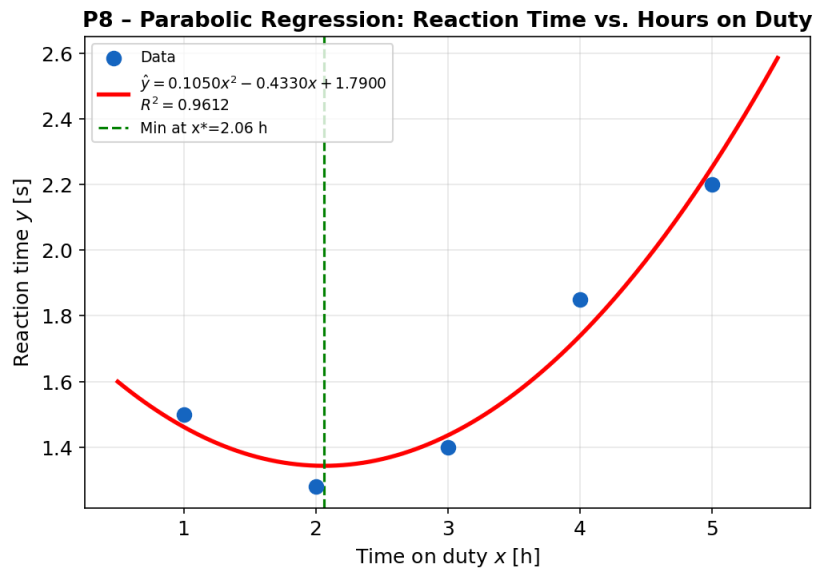


Figura 6: Regresión parabólica del tiempo de reacción. La línea verde indica el mínimo en $x^* \approx 2,06 \text{ h}$.

Problema 9. Regresión Lineal Genérica con Funciones Base Arbitrarias

Planteamiento. Implementar `my_lin_regression(f, x, y)` que calcula β para cualquier conjunto de funciones base $\{f_j\}$.

Desarrollo. La función de estimación es $\hat{y}(x) = \sum_{j=1}^n \beta_j f_j(x) + \beta_{n+1}$ (el término β_{n+1} es el sesgo/intercepto). La matriz de diseño tiene columnas $[f_1(x), f_2(x), \dots, f_n(x), \mathbf{1}]$. El vector de parámetros se obtiene con la fórmula de mínimos cuadrados:

$$\beta = (A^T A)^{-1} A^T y.$$

Implementación.

```

1 def my_lin_regression(f, x, y):
2     x = np.array(x, dtype=float)
3     y = np.array(y, dtype=float)
4     cols = [fi(x) for fi in f] + [np.ones_like(x)]
5     A = np.column_stack(cols)
6     beta = np.linalg.inv(A.T @ A) @ A.T @ y
7     return beta

```

Listing 4: Función my_lin_regression

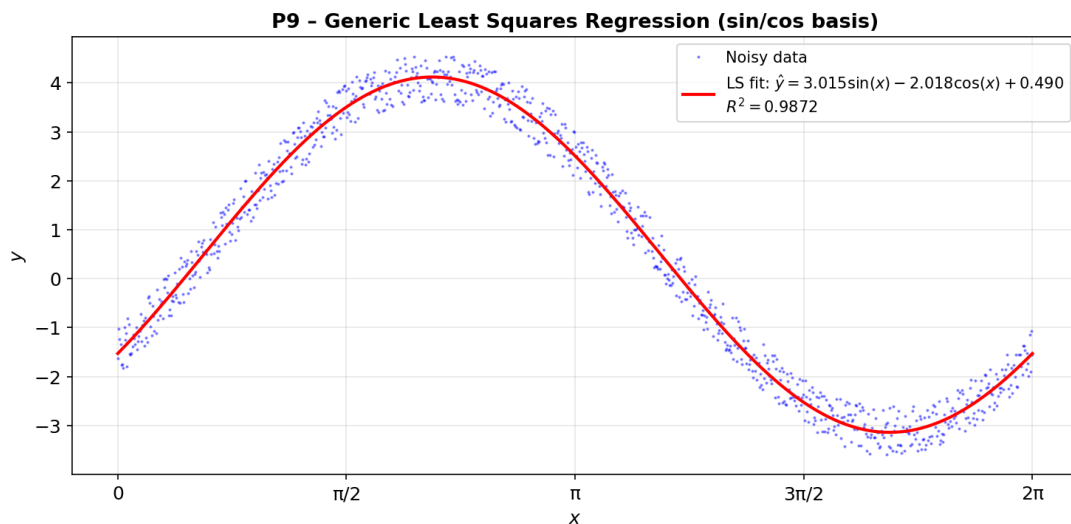
Caso de prueba. Datos generados como $y = 3\sin(x) - 2\cos(x) + \varepsilon$, con $\varepsilon \sim U(0, 1)$, $x \in [0, 2\pi]$, $n = 1000$ puntos. Funciones base: $\{f_1 = \sin, f_2 = \cos\}$.

Cuadro 6: Parámetros recuperados vs. valores esperados

Parámetro	Esperado	Obtenido
β_1 (sin)	3.0	3.0148
β_2 (cos)	-2,0	-2,0183
β_3 (bias)	$\approx 0,5$	0.4903

$$R^2 = 0,9872.$$

Los parámetros recuperados son muy cercanos a los valores teóricos; la pequeña desviación se debe al ruido uniforme añadido.

**Figura 7:** Datos ruidosos (azul) y ajuste por mínimos cuadrados (rojo) con bases sin / cos.

Referencias

- [1] Kong, Q., Siau, T., & Bayen, A. M. (2020). *Python Programming and Numerical Methods: A Guide for Engineers and Scientists*. Elsevier.
- [2] Kreyszig, E. (2006). *Advanced Engineering Mathematics* (10th ed.). John Wiley & Sons.